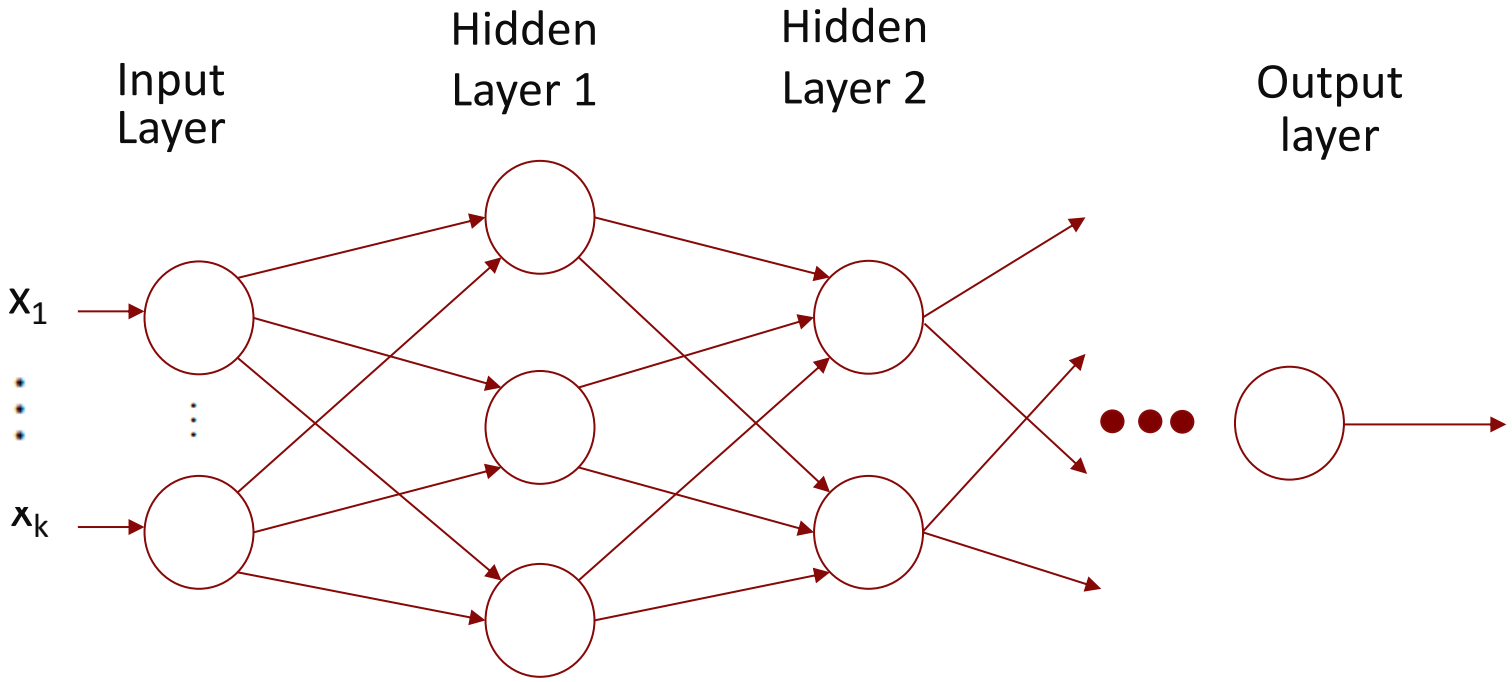
# Training Deep Neural Networks

Neural networks and computer vision

ECE 116 | Fall 2026

Adapted from Farias and Ramakrishnan, MIT OpenCourseWare 15.773, Spring 2024. Changes were made. CC BY-NC-SA 4.0.

# Recap: Designing a DNN




*User chooses the **# of hidden layers**, **# units in each layer**, the **activation function(s)** for the hidden layers and for the output layer*

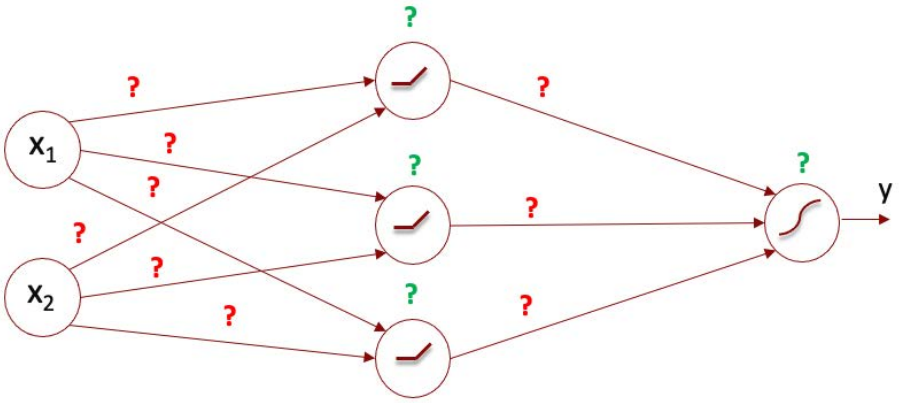
# Training a Deep Neural Network

# Training selects weights and biases

- The architecture defines the model function.
- Training uses labeled examples to adjust the model parameters.
- The objective is to minimize prediction loss.

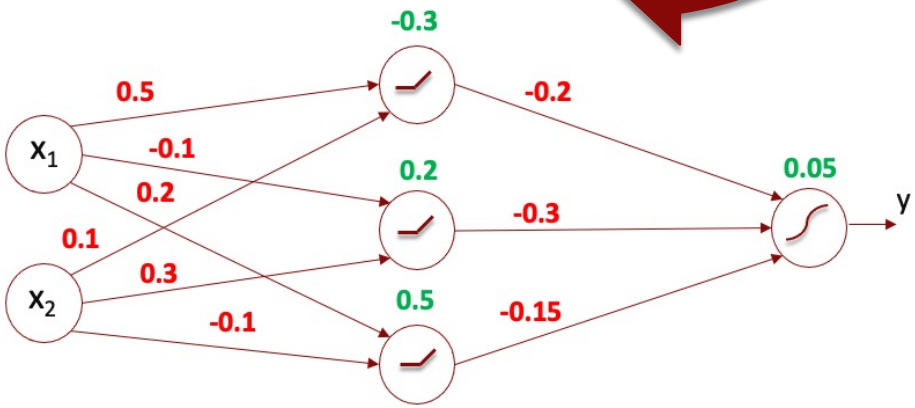
$$\text{data + parameters} \rightarrow \text{predictions} \rightarrow \text{loss}$$

# Training a DNN




+ Data

Training




Training a DNN is no different. It just happens to be a very complex function with lots of parameters.

The essence of training is to find the “best” values for the weights and biases i.e., those that minimize a function that measures the discrepancy between the actual and predicted values

These functions are called loss functions in the DL world

# Loss Functions

# Loss functions

- A “loss function” is a function that quantifies the error in a model’s prediction.
  - If the predictions are close to the actual values, the “loss” would be _____.
  - A perfect model would have a loss of _____.

# Loss functions

- A “loss function” is a function that quantifies the error in a model’s prediction.
  - If the predictions are close to the actual values, the “loss” would be small.
  - A perfect model would have a loss of zero.

# Squared error for numerical predictions

- For a numerical target, squared error is one common loss.

$$\text{error} = \text{prediction} - \text{actual value}$$

$$\text{squared error} = \text{error} \times \text{error}$$

Squaring makes positive and negative errors contribute positively.

# The loss must match the model output

- Numerical prediction: mean squared error
- Binary probability: binary cross-entropy

The next example develops a loss for a predicted probability.

# Mean Squared Error (MSE) Loss is commonly used for general numerical outputs

$$\frac{1}{n}\sum_{i=1}^{n}\left(y^i - model\left(x^i\right)\right)^2$$

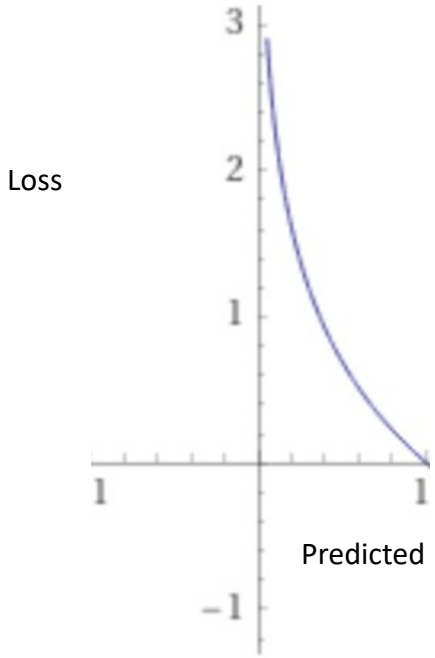
Actual value of $i^{th}$ data point

Predicted value of $i^{th}$ data point

In a Heart Disease Prediction Model the prediction is a probability number and the actual output is 0-1.

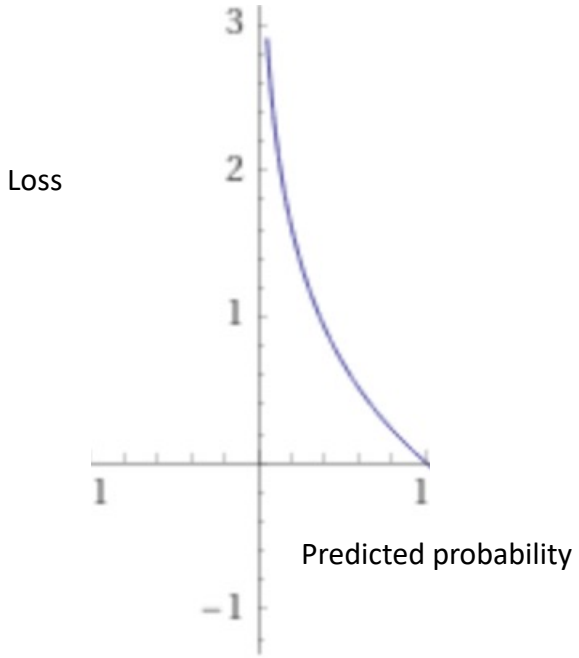
What is a good loss function in this situation?

# For data points with y = 1 (i.e., patients with heart disease), lower predicted probabilities should have higher loss




*For data points with y = 1*

# We can capture this requirement using the log function

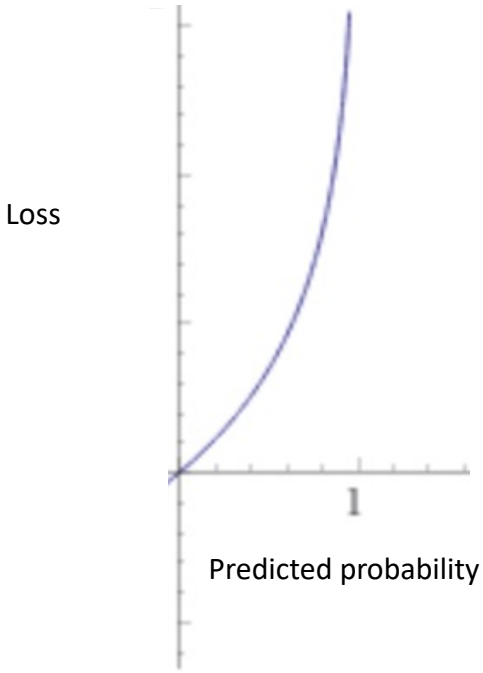



| Predicted probability | *-log(predicted probability)* |
|---|---|
| 1/1000 | 9.97 |
| 1/10 | 3.32 |
| 1/2 | 1.0 |
| 1 | 0.0 |

*For data points with y = 1,*

*loss = -log(predicted probability)*

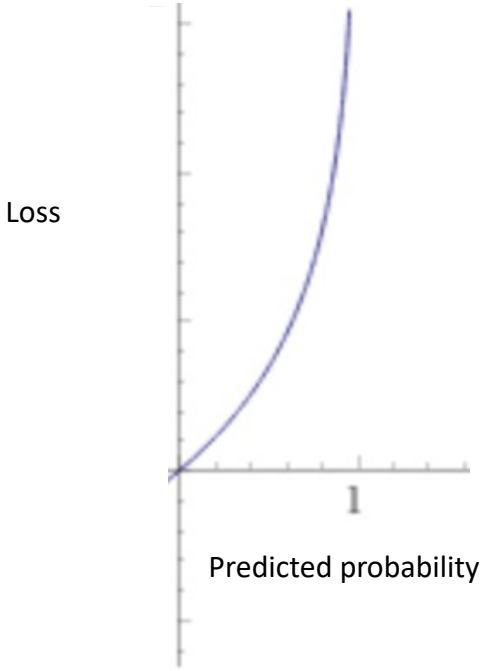
# For data points with y = 0 (i.e., patients without heart disease), higher predicted probabilities should have higher loss




*For data points with y = 0*

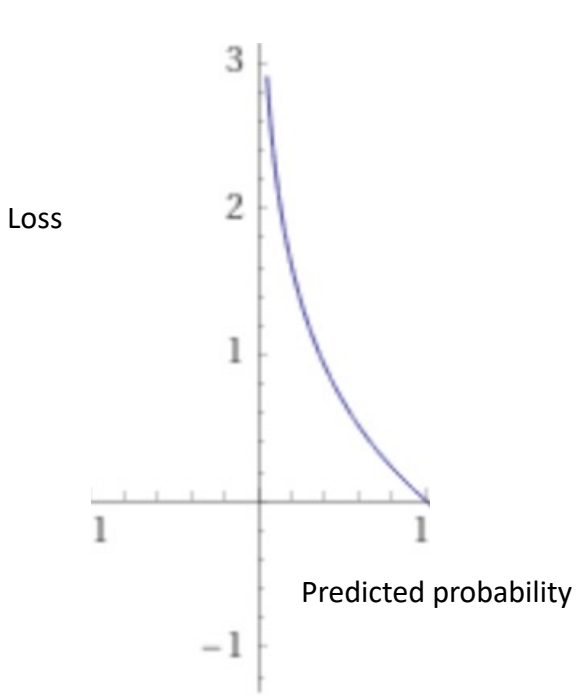
# We can capture this requirement as well using the log function

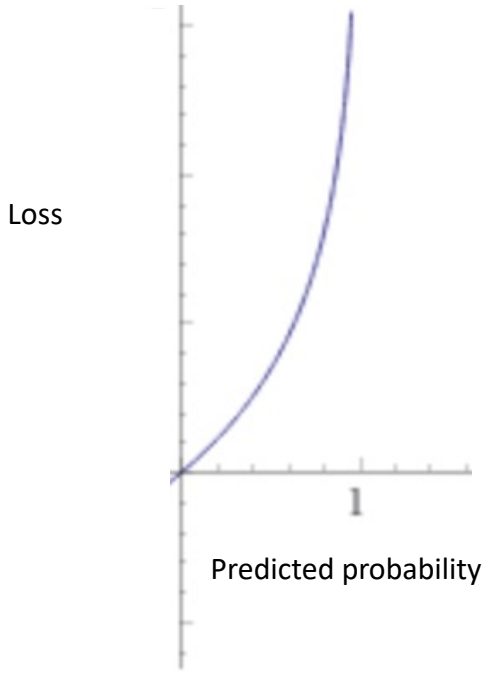
| Predicted probability | *-log(1 - predicted probability)* |
|---|---|
| 1/1000 | 0.001 |
| 1/10 | 0.15 |
| 1/2 | 1.0 |
| 0.999999 | 19.93 |




*For data points with y = 0,*
*loss = -log(1 - predicted probability)*

# Summary




*For data points with y = 1,*
*loss = -log(predicted probability)*

*For data points with y = 0,*
*loss = -log(1 - predicted probability)*

# This can be compactly written as a single expression

*For data points with y = 1,*
*loss = -log(predicted probability)*

*For data points with y = 0,*
*loss = -log(1 - predicted probability)*

$$-y^i \log\left(model\left(x^i\right)\right) - \left(1 - y^i\right)\log\left(1 - model\left(x^i\right)\right)$$

Predicted probability for the $i^{th}$ data point

Predicted probability for the $i^{th}$ data point

# We can now average this across all *n* data points

$$\frac{1}{n}\sum_{i=1}^{n} -y^i \log\left(model\left(x^i\right)\right) - \left(1-y^i\right)\log\left(1-model\left(x^i\right)\right)$$

# This is the Binary Cross-Entropy Loss function!

$$\frac{1}{n}\sum_{i=1}^{n} -y^i \log\left(model\left(x^i\right)\right) - \left(1-y^i\right)\log\left(1-model\left(x^i\right)\right)$$

# Minimizing loss functions

# Minimizing functions

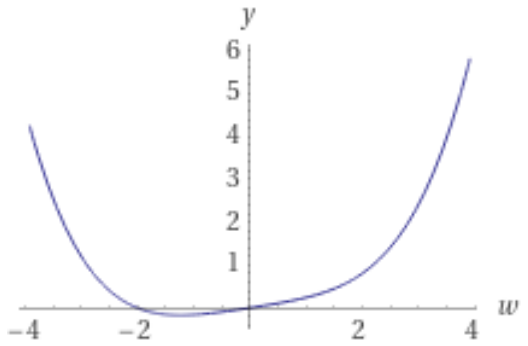
- Loss functions are just a particular kind of function so we will first consider the general problem of minimizing an arbitrary function

- After we develop some intuition about how to do this, we will return to the specific task of minimizing a loss function

# Minimizing a single-variable function

Let's say we want to minimize the function:

$$g(w) = \frac{1}{50}\left(w^4 + w^2 + 10w\right)$$




(w from −3.9 to 3.9)

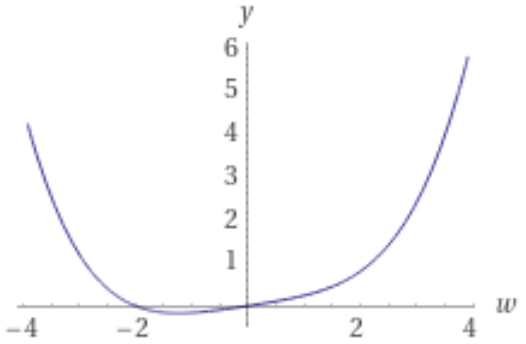
How can we go about this?

https://kenndanielso.github.io/mlrefined/blog_posts/6_First_order_methods/6_4_Gradient_descent.html

# Minimizing a single-variable function

Let's say we want to minimize the function:

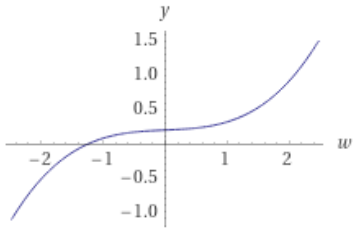
$$g(w) = \frac{1}{50}\left(w^4 + w^2 + 10w\right)$$




(w from −3.9 to 3.9)

Can we use its derivative?

$$\frac{\partial}{\partial w}g\left(w\right) = \frac{2}{25}w^3 + \frac{1}{25}w + \frac{1}{5}$$



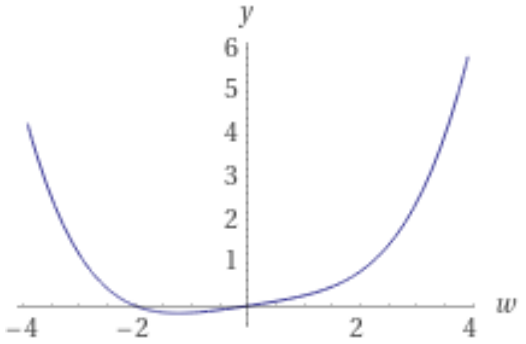

(w from −2.5 to 2.5)

https://kenndanielso.github.io/mlrefined/blog_posts/6_First_order_methods/6_4_Gradient_descent.html

# Minimizing a single-variable function

Let's say we want to minimize the function:

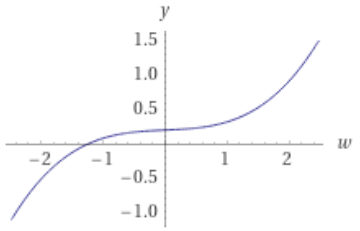
$$g(w) = \frac{1}{50}\left(w^4 + w^2 + 10w\right)$$




(w from −3.9 to 3.9)

What does the derivative at a point tell us?

$$\frac{\partial}{\partial w}g(w) = \frac{2}{25}w^3 + \frac{1}{25}w + \frac{1}{5}$$



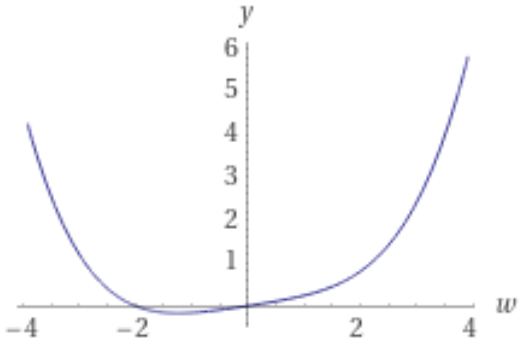

(w from −2.5 to 2.5)

https://kenndanielso.github.io/mlrefined/blog_posts/6_First_order_methods/6_4_Gradient_descent.html

# Minimizing a single-variable function

Let's say we want to minimize the function:

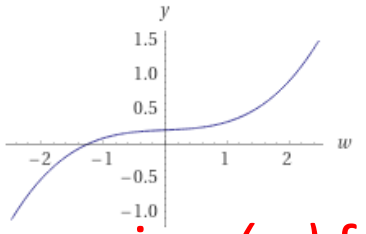
$$g(w) = \frac{1}{50}\left(w^4 + w^2 + 10w\right)$$




(w from −3.9 to 3.9)

What does the derivative at a point tell us?

$$\frac{\partial}{\partial w} g\left(w\right) = \frac{2}{25}w^3 + \frac{1}{25}w + \frac{1}{5}$$



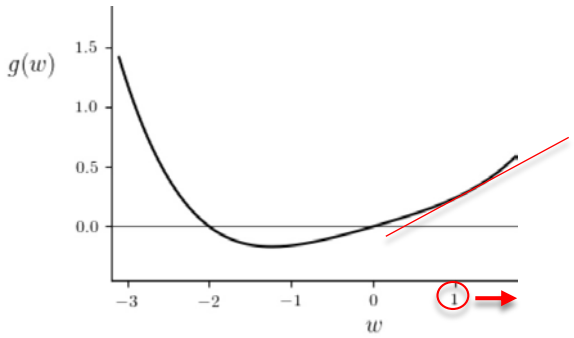

(w from −2.5 to 2.5)

The derivative (or slope) tells us the change in *g(w)* for a small increase in *w*

https://kenndanielso.github.io/mlrefined/blog_posts/6_First_order_methods/6_4_Gradient_descent.html

# The value of knowing the derivative

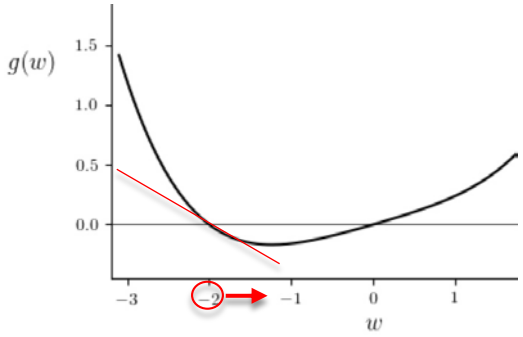
| If the derivative at a point *w* is … | What it means |
|---|---|
| Positive | Increasing *w* slightly will increase *g(w)* |




https://kenndanielso.github.io/mlrefined/blog_posts/6_First_order_methods/6_4_Gradient_descent.html

# The value of knowing the derivative

| If the derivative at a point *w* is … | What it means |
|---|---|
| Positive | Increasing *w* slightly will increase *g(w)* |
| Negative | Increasing *w* slightly will decrease *g(w)* |

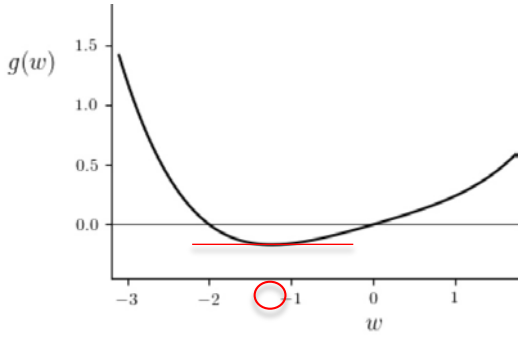





https://kenndanielso.github.io/mlrefined/blog_posts/6_First_order_methods/6_4_Gradient_descent.html

# The value of knowing the derivative

| If the derivative at a point *w* is … | What it means |
|---|---|
| Positive | Increasing *w* slightly will increase *g(w)* |
| Negative | Increasing *w* slightly will decrease *g(w)* |
| ~0 | Changing *w* slightly won't change *g(w)* |




https://kenndanielso.github.io/mlrefined/blog_posts/6_First_order_methods/6_4_Gradient_descent.html

# This suggests an algorithm for minimizing *g(w)*

1. Start with some point *w*

# This suggests an algorithm for minimizing *g(w)*

1. Start with some point *w*
2. Calculate the derivative (i.e., slope) of *g(w)* at *w*

# This suggests an algorithm for minimizing *g(w)*

1. Start with some point *w*
2. Calculate the derivative (i.e., slope) of *g(w)* at *w*

| If the derivative is … | What it means | Since we want to minimize loss, do this … |
|---|---|---|
| Positive | Increasing *w* will increase the loss function | _____ w slightly |
| Negative | Increasing *w* will decrease the loss function | _____ w slightly |
| ~0 | Changing *w* won't change the loss function | _____ |

# This suggests an algorithm for minimizing *g(w)*

1. Start with some point *w*
2. Calculate the derivative (i.e., slope) of *g(w)* at *w*

| If the derivative is … | What it means | Since we want to minimize loss, do this … |
|---|---|---|
| Positive | Increasing *w* will increase the loss function | Reduce w slightly |
| Negative | Increasing *w* will decrease the loss function | Increase w slightly |
| ~0 | Changing *w* won't change the loss function | Stop |

# This suggests an algorithm for minimizing *g(w)*

1. Start with some point *w*
2. Calculate the derivative (i.e., slope) of *g(w)* at *w*

| If the derivative is … | What it means | Since we want to minimize loss, do this … |
|---|---|---|
| Positive | Increasing *w* will increase the loss function | Reduce w slightly |
| Negative | Increasing *w* will decrease the loss function | Increase w slightly |
| ~0 | Changing *w* won't change the loss function | Stop |

3. If you have reached max iterations or run out of time, stop. Else, go to step 2

# This is Gradient Descent!

1. Start with some point $w$
2. Calculate the derivative (i.e., slope) of $g(w)$ at $w$

| If the derivative is … | What it means | Since we want to minimize loss, do this … |
|---|---|---|
| Positive | Increasing $w$ will increase the loss function | Reduce w slightly |
| Negative | Increasing $w$ will decrease the loss function | Increase w slightly |
| ~0 | Changing $w$ won't change the loss function | Stop |

This can be written compactly as

$$w \leftarrow w - \alpha \frac{dg(w)}{dw}$$

3. If you have reached max iterations or run out of time, stop. Else, go to step 2

# Any guesses when Gradient Descent was invented?

# Gradient descent was invented in 1847 by Cauchy!

*Cauchy, A. (1847). Methode generale pour la res- olution des systemes d'equations simultanees. Comptes Rendus de l'Académie des Sciences, 25.* 91




Cauchy around 1840. Lithography by Zéphirin Belliard after a painting by Jean Roller.

Lithograph of Augustin-Louis Cauchy is in the public domain. Source: Wikimedia Commons.

https://en.wikipedia.org/wiki/Augustin-Louis_Cauchy

# Gradient Descent

$$w \leftarrow w - \alpha \frac{dg(w)}{dw}$$

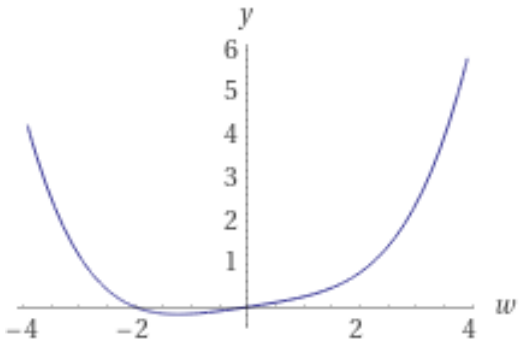
$\alpha$ is called the "learning rate" and is our way of ensuring that we increase or decrease $w$ slightly

Typically set to small values (e.g., 0.1, 0.001, 0.0001) and determined by trial and error

# Let's apply this algorithm to *g(w)*

$$g(w) = \frac{1}{50}\left(w^4 + w^2 + 10w\right)$$

$$\frac{\partial}{\partial w} g(w) = \frac{2}{25}w^3 + \frac{1}{25}w + \frac{1}{5}$$

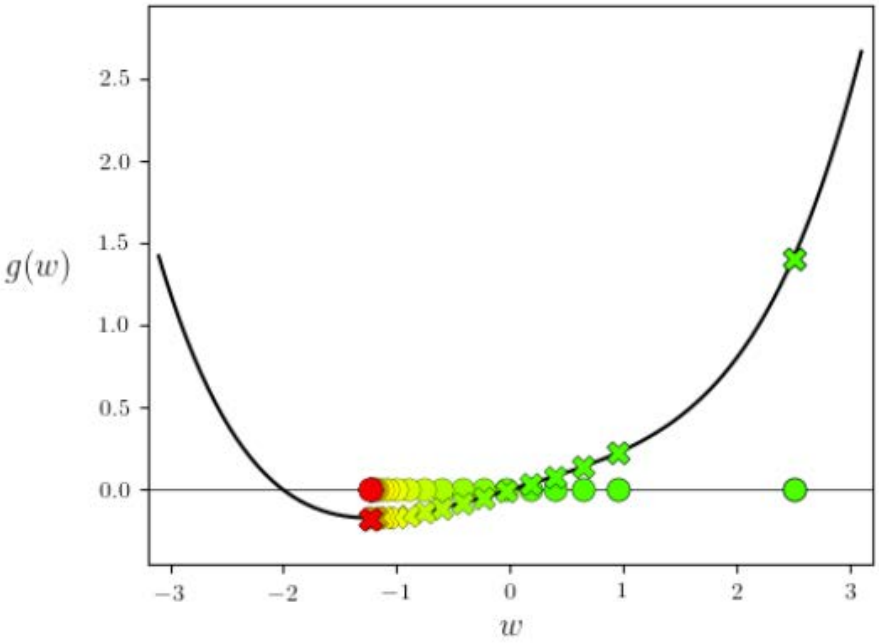



(*w* from −3.9 to 3.9)

$$w \leftarrow w - \alpha \frac{dg(w)}{dw}$$

https://kenndanielso.github.io/mlrefined/blog_posts/6_First_order_methods/6_4_Gradient_descent.html

# Gradient Descent in action




We will start at $w = 2.5$, set $\alpha = 1$ and run the algorithm. In a few iterations, it finds the minimum.

https://kenndanielso.github.io/mlrefined/blog_posts/6_First_order_methods/6_4_Gradient_descent.html

# Minimizing a multi-variable function

$$g(w_1, w_2) = {w_1}^2 + {w_2}^2 + 2$$

We can calculate the partial derivative of $g(w_1, w_2)$

$$\left[\frac{\partial g}{\partial w_1}, \frac{\partial g}{\partial w_2}\right] = [2w_1, 2w_2]$$

How should we interpret this?

https://kenndanielso.github.io/mlrefined/blog_posts/6_First_order_methods/6_4_Gradient_descent.html

# Minimizing a multi-variable function

$$g(w_1, w_2) = {w_1}^2 + {w_2}^2 + 2$$

$$\nabla g = \left[\frac{\partial g}{\partial w_1}, \frac{\partial g}{\partial w_2}\right] = [2w_1, 2w_2]$$

The first number is the change in *g(w)* for a small increase in $w_1$, *with* $w_2$ *kept unchanged.* The second number is the change in *g(w)* for a small increase in $w_2$, *with* $w_1$ *kept unchanged*

https://kenndanielso.github.io/mlrefined/blog_posts/6_First_order_methods/6_4_Gradient_descent.html

# Minimizing a multi-variable function

$$g(w_1, w_2) = {w_1}^2 + {w_2}^2 + 2$$

$$\nabla g = \left[\frac{\partial g}{\partial w_1}, \frac{\partial g}{\partial w_2}\right] = [2w_1, 2w_2]$$

The first number is the change in *g(w)* for a small increase in $w_1$, *with* $w_2$ *kept unchanged.* The second number is the change in *g(w)* for a small increase in $w_2$, *with* $w_1$ *kept unchanged*

This is called the "gradient" of $g(w_1, w_2)$ and written as $\nabla g$

https://kenndanielso.github.io/mlrefined/blog_posts/6_First_order_methods/6_4_Gradient_descent.html

# Minimizing a multi-variable function

$$\nabla g = \left[\frac{\partial g}{\partial w_1}, \frac{\partial g}{\partial w_2}\right] = [2w_1, 2w_2]$$

We can simply do gradient descent on each coordinate by using the corresponding partial derivative.

$$w_1 \leftarrow w_1 - \alpha \left(\frac{\partial g}{\partial w_1}\right)$$

$$w_2 \leftarrow w_2 - \alpha \left(\frac{\partial g}{\partial w_2}\right)$$

https://kenndanielso.github.io/mlrefined/blog_posts/6_First_order_methods/6_4_Gradient_descent.html

# Minimizing a multi-variable function

$$\nabla g = [2w_1, 2w_2]$$

$$w_1 \leftarrow w_1 - \alpha \left(\frac{\partial g}{\partial w_1}\right)$$

$$w_2 \leftarrow w_2 - \alpha \left(\frac{\partial g}{\partial w_2}\right)$$

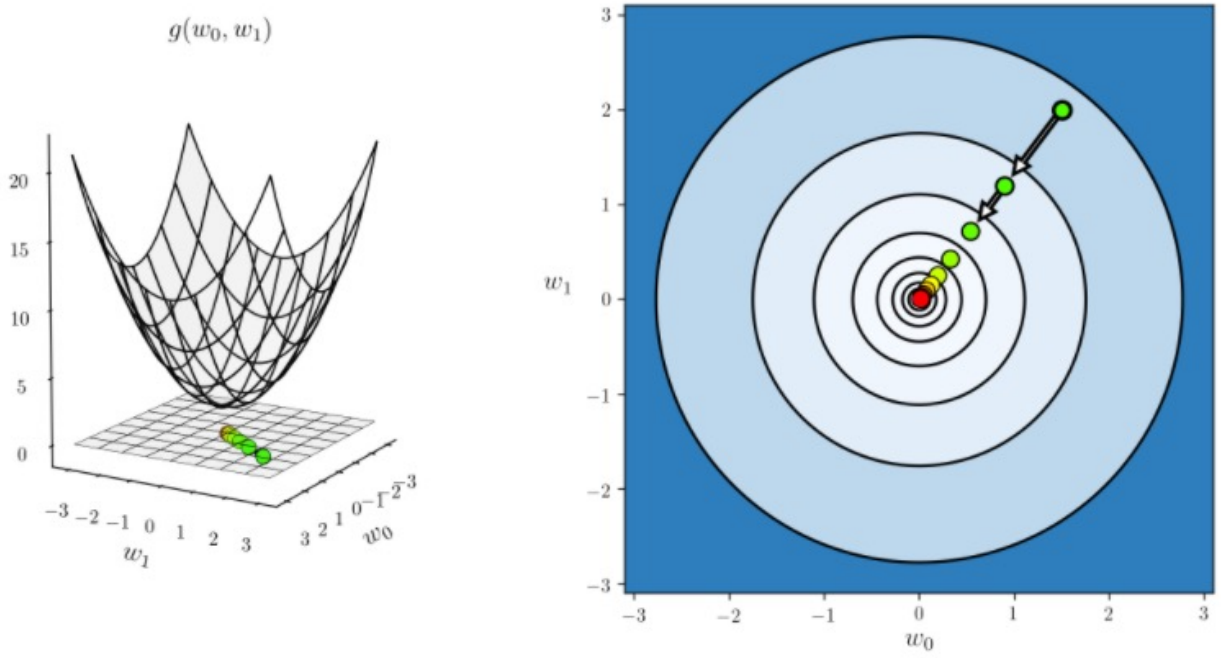
As before, this whole thing can be summarized compactly as:

$$w \leftarrow w - \alpha \nabla g(w)$$

https://kenndanielso.github.io/mlrefined/blog_posts/6_First_order_methods/6_4_Gradient_descent.html

# Gradient Descent in two dimensions

$$g(w_0, w_1) = {w_0}^2 + {w_1}^2 + 2$$




https://kenndanielso.github.io/mlrefined/blog_posts/6_First_order_methods/6_4_Gradient_descent.html

Gradient descent figures © Jeremy Watt/neonwatty on GitHub. All rights reserved. This content is excluded from our Creative Commons license. For more information, see https://ocw.mit.edu/help/faq-fair-use.

GD may stop near a local minimum (not necessarily a global minimum) or a saddle point but we don’t worry about this in practice.

# Minimizing a loss function with gradient descent

*Minimize* $\frac{1}{n}\sum_{i=1}^{n} -y^i \log\left(model\left(x^i\right)\right) - \left(1-y^i\right)\log\left(1-model\left(x^i\right)\right)$

What are the variables we need to change to minimize this function?

# Minimizing a loss function with gradient descent

*Minimize* $\frac{1}{n}\sum_{i=1}^{n} -y^i \log\left(model\left(x^i\right)\right) - \left(1-y^i\right)\log\left(1-model\left(x^i\right)\right)$

What are the variables we need to change to minimize this function?

They are the parameters "hiding" inside $model(x_i)$

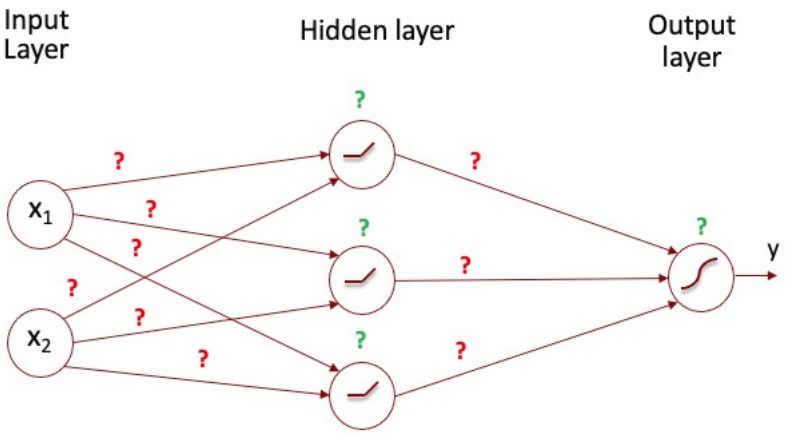
# Minimizing a loss function with gradient descent

*Minimize*

$$\frac{1}{n}\sum_{i=1}^{n} -y^i \log\left(model\left(x^i\right)\right) - \left(1-y^i\right)\log\left(1-model\left(x^i\right)\right)$$

$$model(x^i) = \frac{1}{1+e^{-(w_1+w_2 max(0,w_3+w_4 x_1^i+w_5 x_2^i)+w_6 max(0,w_7+w_8 x_1^i+w_9 x_2^i)+w_{10} max(0,w_{11}+w_{12} x_1^i+w_{13} x_2^i))}}$$

Recall this model and the NN it represents



# Minimizing a loss function with gradient descent

*Minimize*

$$\frac{1}{n}\sum_{i=1}^{n} -y^i \log\left(model\left(x^i\right)\right) - \left(1-y^i\right)\log\left(1-model\left(x^i\right)\right)$$

$$model(x^i) = \frac{1}{1+e^{-(w_1+w_2 max(0,w_3+w_4 x_1^i+w_5 x_2^i)+w_6 max(0,w_7+w_8 x_1^i+w_9 x_2^i)+w_{10} max(0,w_{11}+w_{12} x_1^i+w_{13} x_2^i))}}$$

$w_1$, $w_2$, …, $w_{13}$ are the variables we can change to minimize the loss function

The values of $x_1$, $x_2$ and y, on the other hand, are just data

# Minimizing a loss function with gradient descent

*Minimize*

$$\frac{1}{n}\sum_{i=1}^{n} -y^i \log\left(model\left(x^i\right)\right) - \left(1-y^i\right)\log\left(1-model\left(x^i\right)\right)$$

$$model(x^i) = \frac{1}{1+e^{-(w_1+w_2 max(0,w_3+w_4 x_1^i+w_5 x_2^i)+w_6 max(0,w_7+w_8 x_1^i+w_9 x_2^i)+w_{10} max(0,w_{11}+w_{12} x_1^i+w_{13} x_2^i))}}$$

Imagine replacing $model\left(x^i\right)$ with the mathematical expression above wherever $model\left(x^i\right)$ appears in the loss function

Now, your loss function is just a ”good old” function of $w_1$, $w_2$, …, $w_{13}$ and you can apply gradient descent to it as we normally would.

# Backpropagation (aka ‘backprop’)

- Backpropagation is an efficient way to compute the gradient of the loss function
- The efficiency stems from exploiting the layer-by-layer architecture of NNs

# Backpropagation (aka ‘backprop’)

- Backpropagation is an efficient way to compute the gradient of the loss function
- The efficiency stems from exploiting the layer-by-layer architecture of NNs
- By organizing the computation in the form of a “computational graph”, we can incrementally calculate the gradient one layer at a time using matrix multiplications (and other simple operations). This approach also eliminates redundant calculations

# Backpropagation (aka ‘backprop’)

- Backpropagation is an efficient way to compute the gradient of the loss function
- The efficiency stems from exploiting the layer-by-layer architecture of NNs
- By organizing the computation in the form of a “computational graph”, we can incrementally calculate the gradient one layer at a time using matrix multiplications (and other simple operations). This approach also eliminates redundant calculations
- It turns out that Graphic Processing Units (GPUs), originally invented to speed up video games, are perfectly suited for matrix multiplications!

# Backpropagation and GPUs

- Backpropagation computes the loss gradient layer by layer.
- The calculation reuses intermediate results.
- GPUs execute the required matrix operations efficiently.

Together, these make large neural networks practical to train.

For a walk through of the math:
https://youtu.be/Ilg3gGewQ5U?si=xKRJ7hsHJY6YQF-A
https://youtu.be/tIeHLnjs5U8?si=k1brRdeG_YMXegxC

# Gradient Descent → Stochastic Gradient Descent

# Making Gradient Descent work with large datasets

- Problem: For large datasets (e.g., *n* in the millions), computing the gradient of the loss function can be very expensive

# Making Gradient Descent work with large datasets

- Problem: For large datasets (e.g., *n* in the millions), computing the gradient of the loss function can be very expensive

$$w \leftarrow w - \alpha \nabla g(w)$$

- The Solution:
  - At each iteration, instead of using all the n data points in the calculation of the gradient of the loss function, randomly choose just a few of the n observations (called a *minibatch*) and use only these observations to compute the partial derivatives.

# Making Gradient Descent work with large datasets

- Problem: For large datasets (e.g., *n* in the millions), computing the gradient of the loss function can be very expensive

- The Solution:
  - At each iteration, instead of using all the n data points in the calculation of the gradient of the loss function, randomly choose just a few of the n observations (called a *minibatch*) and use only these observations to compute the partial derivatives.
  - This is called Stochastic Gradient Descent (SGD)*

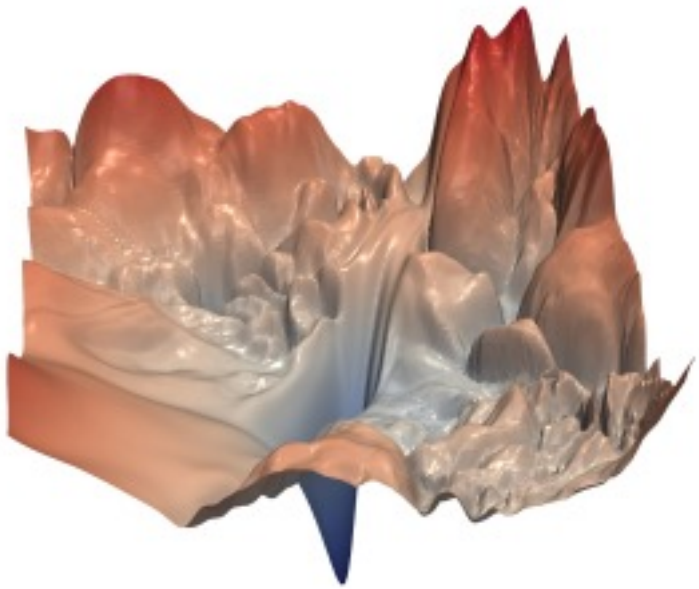
* Strictly speaking, SGD chooses just one observation. What we are describing here is Minibatch Gradient Descent but the term SGD is widely used in the field to describe the latter so we will do the same

# Minibatch stochastic gradient descent

- A full gradient can be expensive on a large dataset.
- Each update uses a randomly selected minibatch.
- The minibatch gradient approximates the full-data gradient.

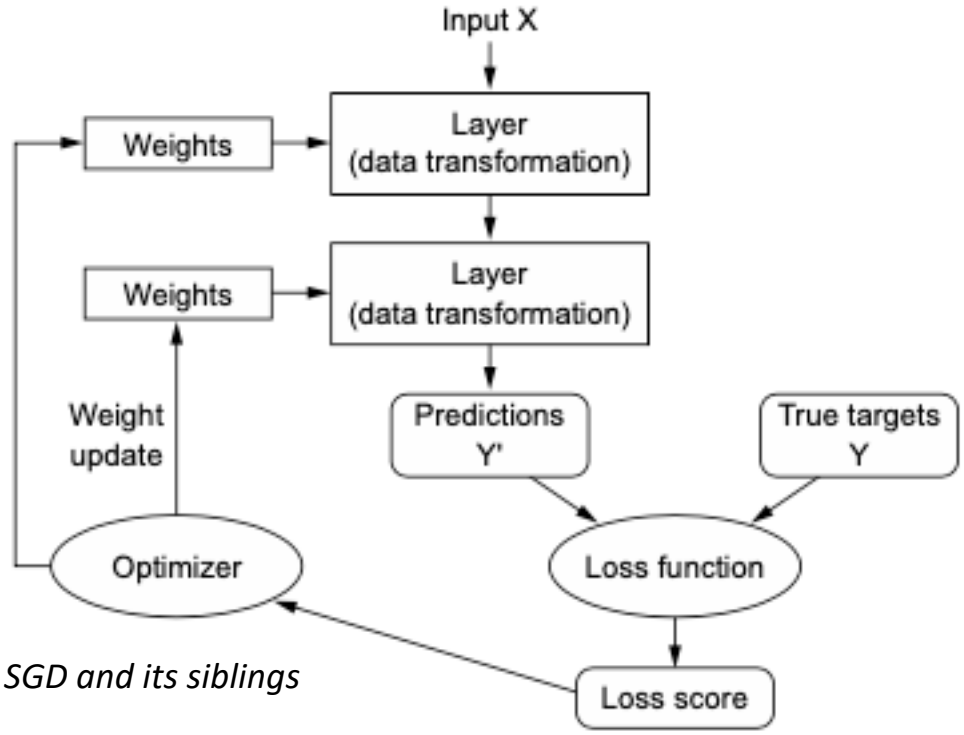
ECE116 uses Adam as the default optimizer.

# Visualization of an actual DL loss function landscape



https://arxiv.org/pdf/1712.09913.pdf

# Summary of overall training flow




*SGD and its siblings*

**Figure 2.26 Relationship between the network, layers, loss function, and optimizer**

# Source and license

**This lecture adapts material from:**

Vivek Farias and Rama Ramakrishnan, Hands-On Deep Learning,
MIT OpenCourseWare 15.773, Spring 2024.

**Source:**

ocw.mit.edu/courses/15-773-hands-on-deep-learning-spring-2024/

License: Creative Commons Attribution-NonCommercial-ShareAlike 4.0
creativecommons.org/licenses/by-nc-sa/4.0/

Adapted for ECE116 at UC San Diego, Fall 2026.
Changes include course framing, sequencing, omissions, and ECE116 branding.